

Réplica Simplificada do Akinator

Uma aplicação interativa que transforma a teoria de [árvores de decisão](#) e [sistemas especialistas](#) em um jogo funcional — usando HTML, CSS e JavaScript puro.

IA CLÁSSICA

ÁRVORES DE DECISÃO

JAVASCRIPT

Ana Carolina Lino e Octávio Lamounier





O que é o Akinator?

Lançado em 2007, o Akinator foi um dos primeiros fenômenos virais a popularizar a IA no cotidiano. O jogador pensa em um personagem — real, fictício ou animal — e o sistema faz perguntas para adivinhá-lo em poucos passos.

Sistemas Especialistas

Motor Limule com regras lógicas, árvore de decisão e cálculo de probabilidades

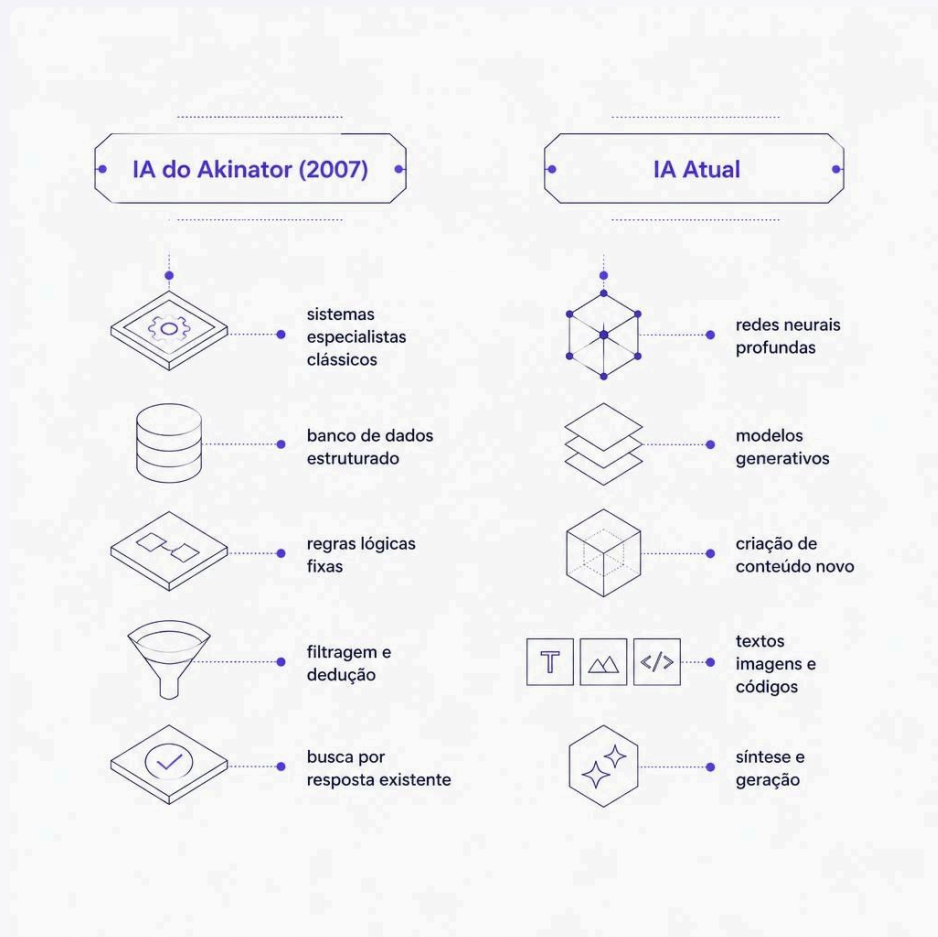
Subjetividade

Não elimina candidatos, ele aumenta o peso (a confiança)

Machine Learning

Se acerta, reforça os contadores. Se ele erra, solicita o personagem e ajustando os pesos com base nessa nova informação.

IA do Akinator × IA Atual



A diferença fundamental

A IA do Akinator foi projetada para **analisar e deduzir** — ela filtra um banco de dados existente usando regras lógicas e probabilidade.

As IAs modernas, como o ChatGPT, são projetadas para **sintetizar e criar** — gerando conteúdo inédito a partir de padrões aprendidos.

☐ Ambas são Inteligência Artificial. A diferença está no objetivo.

Produção do Projeto

O desenvolvimento seguiu etapas planejadas para garantir clareza e evitar retrabalho — do papel ao código.

01

Elaboração das Perguntas

Definição de todas as perguntas do jogo

02

Seleção dos Personagens

20 heróis da Marvel e DC com caminhos únicos

03

Árvore Visual

Diagrama das ramificações antes do código

04

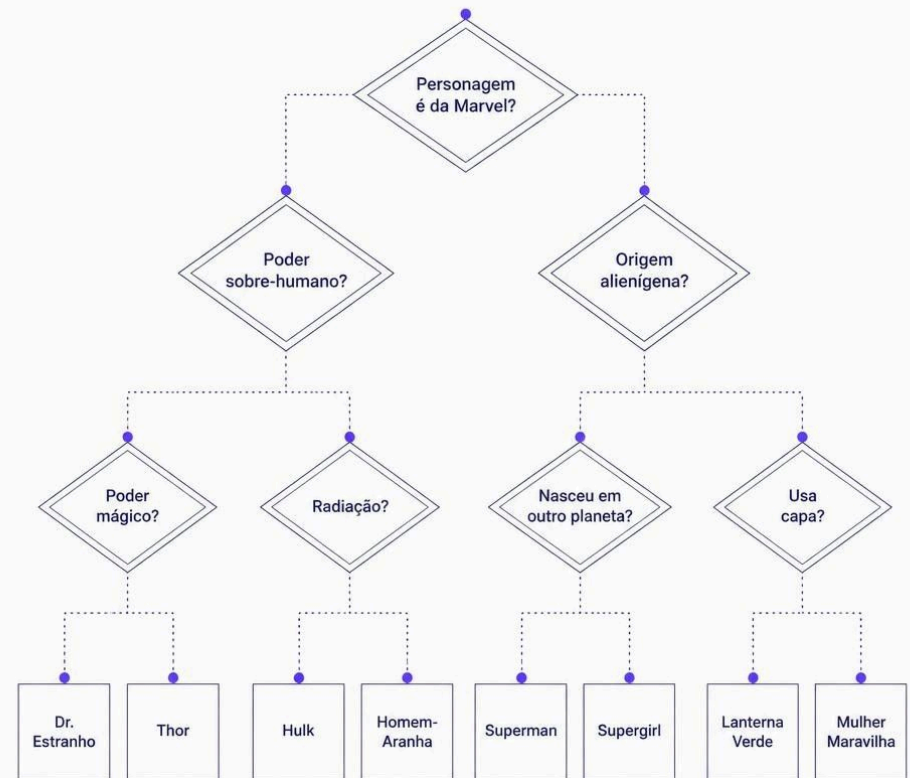
Stack Tecnológica

HTML, CSS e JavaScript puro — sem frameworks

Arquitetura: A Árvore de Decisão

Uma **árvore binária estática** organiza o raciocínio do jogo. Cada pergunta é um nó com dois caminhos: **Sim** → **esquerda** e **Não** → **direita**.

- **Raiz:** primeira pergunta do jogo
- **Nós:** decisão intermediária
- **Folhas:** resultado final com o herói



Estrutura dos Nós em Código

Duas classes em JavaScript com Orientação a Objetos representam toda a árvore.

Classe No — Perguntas

Armazena o texto, a imagem do gênio e os ponteiros para os caminhos.

```
class No {  
  constructor(valor, imagem,  
    esquerda, direita) {  
    this.valor = valor;  
    this.imagem = imagem;  
    this.esquerda = esquerda;  
    this.direita = direita;  
  }  
}
```

Classe NoFolha — Resultados

Herda de No e representa o palpite final com o GIF do herói.

```
class NoFolha extends No {  
  constructor(valor, imagem) {  
    super(valor);  
    this.imagem = imagem;  
  }  
}  
  
const doctorStranger =  
  new NoFolha(  
    "É o Dr. Estranho!",  
    "gifs/DoctorStranger.gif"  
  );
```

Ciclo do Jogo



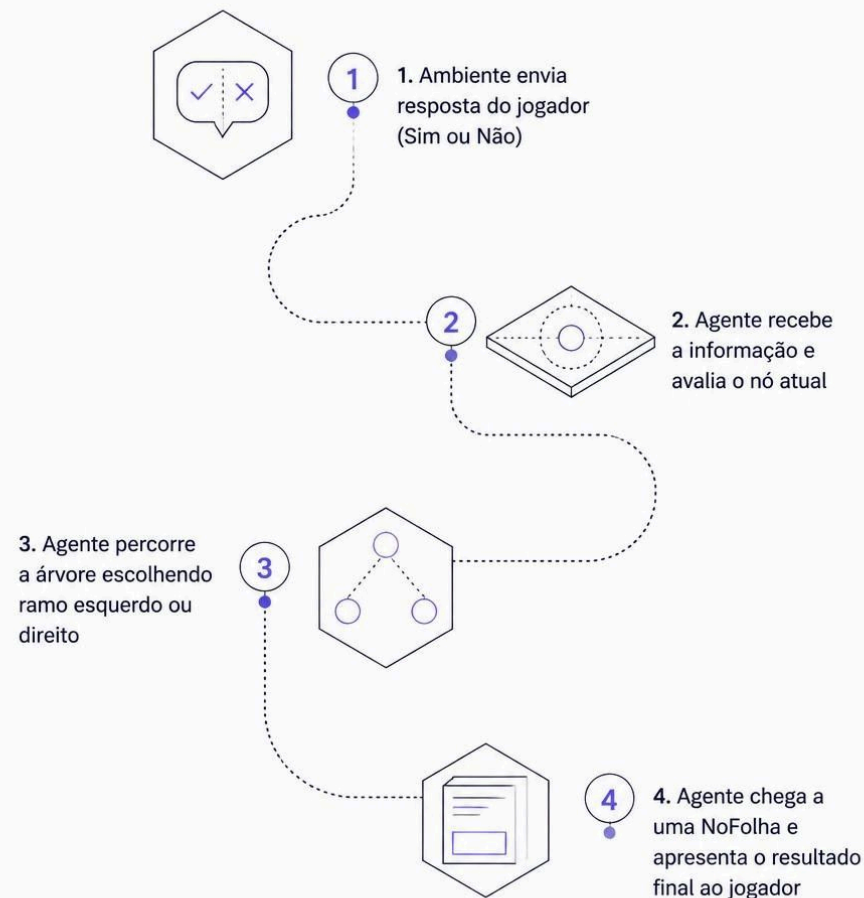
O ciclo é controlado por quatro funções: `iniciarJogo()`, `responder()`, `atualizarTela()` e `reiniciarJogo()`. A função `responder()` é o motor central — ela move `noAtual` pela árvore com base no clique do jogador.

Árvore + Agente Inteligente

A árvore representa o **processo de raciocínio** do agente. A cada resposta, o agente decide: fazer nova pergunta ou revelar o resultado.

- **Sim** → `noAtual.esquerda`
- **Não** → `noAtual.direita`
- **NoFolha** → palpite final

❏ Esta é uma simplificação de um agente inteligente — sem aprendizado contínuo, mas com decisão lógica estruturada.



Exemplo e Limitações

Exemplo: Dr. Estranho

01

Herói da Marvel?

→ Sim

02

Poder sobre-humano?

→ Sim

03

Poder mágico?

→ Sim

04

Estudou magia?

→ Sim → Dr. Estranho!

Limitações do Modelo

- Base de personagens **estática** (hardcoded)
- Sem aprendizado automático
- Respostas apenas **Sim / Não**
- Um erro desvia o caminho sem recuperação
- Ordem das perguntas é fixa

O Akinator original é **adaptativo e probabilístico** — nossa réplica é determinística.

Conclusão

Esta réplica transforma teoria abstrata em aplicação real — combinando [Orientação a Objetos](#), manipulação do DOM e árvores binárias para reproduzir a lógica do Akinator.

Teoria Aplicada

Árvores binárias e sistemas especialistas em ação

Código Elegante

Classes limpas, hierarquia clara e DOM dinâmico

Base para Evolução

JSON externo, localStorage e probabilidades são os próximos passos

🎮 **Desafio:** Escolha mentalmente um dos 20 heróis e veja se nosso Akinator consegue adivinhar!



Mais informações

GitHub:

[OctavioSantosLamounier](#)

[Carolzita001](#)